

**PROJECT
ON
CAMPUSMATE**
**INTRODUCTION TO
PROBLEMSOLVING AND PROGRAMMING
(CSE 1021)**

Student Productivity System

A Command-Line Academic Management Application

Submitted in partial fulfilment of the requirements for the academic project

Student Name: Pentakota. Hemanth

Roll No.: 26MIM10061

College: VIT BHOPAL

Academic Year: 2026–2027

Submitted to: Dr. Adarsh Patel

Slot: A11+A12+A13+A14+D11+D12

CERTIFICATE

This is to certify that the project entitled “Campus Mate – Student Productivity System” has been prepared as an academic project. The work describes a command-line Python application for managing common academic activities including student profile information, assignments, attendance, study planning, academic performance, and dashboard reports.

The project is organized into separate modules and uses SQLite for local data persistence. The submitted report has been prepared from the project documentation, requirements, problem statement, and program entry-point files supplied for the project.

Student Name: Pentakota. Hemanth

DECLARATION

I hereby declare that the project report entitled “Campus Mate – Student Productivity System” is prepared for academic purposes. The report presents the problem statement, objectives, methodology, implementation structure, functional requirements, testing approach, results discussion, conclusion, and future enhancement possibilities of the Campus Mate application.

The project uses Python, SQLite, the Python standard library, and unit test. The report is based on the project files supplied for this work.

Student Signature: *P. Hemanth*

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, faculty members, department, and institution for providing the opportunity to undertake this academic project.

I acknowledge the role of Python and SQLite in implementing the Campus Mate application. The modular structure of the project helped organize student management, assignments, attendance, study planning, academic performance, reporting, validation, and database operations.

I also thank everyone who provided guidance and feedback during the development and documentation of the project.

ABSTRACT

Campus Mate is a command-line Python application designed to combine several common academic management activities into one system. The supplied project documentation states that the application stores a student profile, assignments, attendance, study sessions, and academic performance using SQLite.

The system provides a Student Profile, Assignment Manager, Attendance Manager, Study Planner, Academic Performance, and Dashboard & Reports through a menu-driven command-line interface. Assignment management includes adding, viewing, updating, deleting, searching, completing, and listing upcoming assignments. Attendance functionality calculates attendance, provides warnings below 75%, and calculates classes needed to reach 75%. The study planner manages study sessions and summaries, while academic performance stores marks and calculates totals and grades.

The application uses Python 3, SQLite, the Python standard library, and unit test. The supplied requirements file specifies that no external Python packages are required. The modular design supports maintainability, local data persistence, validation, and error handling.

TABLE OF CONTENTS

Chapter 1: Introduction

- 1.1 Background
- 1.2 Objectives of the Work and Contribution
- 1.3 Scope of the Project
- 1.4 Target Users

Chapter 2: Literature Review

- 2.1 Existing Academic Task Management Approach
- 2.2 Problem Statement
- 2.3 Research/Project Gap Summary
- 2.4 Proposed Solution

Chapter 3: Research Methodology

- 3.1 Development Approach
- 3.2 Requirement Analysis
- 3.3 System Architecture
- 3.4 Database and Data Persistence
- 3.5 Validation and Error Handling
- 3.6 Testing Methodology

Chapter 4: Implementation

- 4.1 Main Program and Menu
- 4.2 Student Profile Module
- 4.3 Assignment Manager
- 4.4 Attendance Manager
- 4.5 Study Planner
- 4.6 Academic Performance
- 4.7 Dashboard and Reports
- 4.8 Project Structure

Chapter 5: Results and Discussion

- 5.1 Implemented Functionalities
- 5.2 Requirements Coverage

5.3 Testing and Expected Behaviour

5.4 Discussion

Chapter 6: Conclusion and Future Work

6.1 Conclusion

6.2 Future Enhancements

References

Appendix – 1: Project Files and Execution

Appendix – 2: Main Program Listing Summary

Chapter 1: Introduction

1.1 Background

Students often manage assignments, attendance, study schedules, and marks using separate notes or applications. According to the supplied problem statement, this separation can make it harder to track academic tasks and identify important issues such as pending assignments or attendance below the required level.

CampusMate addresses this stated problem by combining these academic activities into one command-line Python application. The system uses SQLite to persist information locally and presents the functionality through clear menus.

1.2 Objectives of the Work and Contribution

- Provide a single application for common academic management activities.
- Allow creation and viewing of a student profile.
- Provide assignment CRUD operations and search functionality.
- Track attendance and calculate attendance percentages.
- Calculate the additional classes needed to reach a 75% attendance target.
- Create and manage study sessions and summarize study hours.
- Record marks and calculate totals and grades.
- Provide a combined dashboard with academic status and alerts.
- Use validation, error handling, modular code, and local SQLite persistence.
- Provide automated unit tests for important calculations and input-validation logic.

1.3 Scope of the Project

The current version covers student profile management, assignment CRUD and search, attendance tracking and percentage calculation, attendance target calculation for 75%, study session planning, marks and grade calculation, dashboard and alerts, and automated unit tests.

1.4 Target Users

The primary target user is a college student who wants a simple academic management tool that can be run from a computer terminal.

Chapter 2: Literature Review

2.1 Existing Academic Task Management Approach

The supplied project statement describes a common situation in which students use separate notes or applications for assignments, attendance, study schedules, and marks. The project therefore treats fragmentation of academic information as the practical background for the proposed system.

2.2 Problem Statement

Students often manage assignments, attendance, study schedules and marks using separate notes or applications. This makes it harder to track academic tasks and identify important issues such as pending assignments or attendance below the required level.

2.3 Research/Project Gap Summary

For the scope of this academic project, the identified gap is the lack of a single lightweight local tool that combines the listed student academic activities in one menu-driven application. CampusMate narrows this gap by bringing the activities together while retaining simple terminalbased operation.

2.4 Proposed Solution

CampusMate is proposed as a command-line Python system that combines academic activities into one application. It stores data in SQLite and provides menus for managing academic information. The solution emphasizes usability, maintainability, local persistence, validation, and lightweight resource usage.

Chapter 3: Research Methodology

3.1 Development Approach

The project follows a modular software-development approach. The main program initializes the database and then presents the main menu. Individual menu functions are imported from separate modules for student profile, assignments, attendance, study planning, performance, and reports.

3.2 Requirement Analysis

Functional requirements were defined around the core academic tasks. The user shall be able to create and view a student profile; add, view, update, delete, and search assignments; record attendance and calculate percentages; create and manage study sessions; record marks and calculate total and grade; and view a combined dashboard with alerts.

3.3 System Architecture

The supplied main.py shows the application entry point importing seven functional components: assignments, attendance, database, performance, reports, student, and study planner. The main() function first calls initialize_database() and then starts main_menu().

3.4 Database and Data Persistence

SQLite is used as the local database technology. The project README states that the program automatically creates the data folder and campusmate.db file. This supports persistence so that information remains available after the program closes.

3.5 Validation and Error Handling

The requirements specify that validation prevents many invalid inputs and that invalid values are rejected with useful messages. A dedicated validation module is included in the project structure, supporting separation of validation logic from the feature modules.

3.6 Testing Methodology

The project uses Python unittest. The README states that the test suite checks attendance calculations, performance calculations, and input-validation logic. The documented test command is `python -m unittest discover -s tests -v`.

Chapter 4: Implementation

4.1 Main Program and Menu

The main program displays the CampusMate banner and presents seven choices: Student Profile, Assignment Manager, Attendance Manager, Study Planner, Academic Performance, Dashboard & Reports, and Exit. The selected option calls the corresponding module function. Database initialization occurs before the main menu starts.

4.2 Student Profile Module

The Student Profile component is responsible for creating and viewing profile information. This gives the system a basic student identity record around which the other academic activities can be organized.

4.3 Assignment Manager

The Assignment Manager supports adding, viewing, updating, deleting, searching, completing, and listing upcoming assignments. These operations provide the CRUD and search functionality defined in the project scope.

4.4 Attendance Manager

The Attendance Manager records attendance, calculates attendance percentage, shows warnings when attendance is below 75%, and calculates classes needed to reach the 75% target.

4.5 Study Planner

The Study Planner allows study sessions to be created, marked complete, deleted, and summarized in terms of study hours.

4.6 Academic Performance

The Academic Performance module stores marks and calculates totals and grades. This provides a simple way to keep academic performance information together with the other student records.

4.7 Dashboard and Reports

The Dashboard & Reports component provides a combined academic status view and alerts. It brings information from the different academic areas into a single reporting function.

4.8 Project Structure

- main.py – program entry point and main menu
- src/database.py – SQLite connection and table creation

- `src/validation.py` – input validation helpers
- `src/student.py` – student profile module
- `src/assignments.py` – assignment module
- `src/attendance.py` – attendance module
- `src/study_planner.py` – study planning module
- `src/performance.py` – academic performance module
- `src/reports.py` – dashboard and alerts
- `tests/` – automated tests
- `README.md` – project overview, features, technologies, requirements, execution, testing, and structure
- `requirements.txt` – confirms that no external Python packages are required

Chapter 5: Results and Discussion

5.1 Implemented Functionalities

Based on the supplied project documentation and main program, the implemented application is organized around six user-facing academic areas plus exit: profile management, assignment management, attendance, study planning, performance, and dashboard/reporting.

Module	Main Functions	Purpose
Student Profile	Create, view	Manage student profile information
Assignment Manager	Add, view, update, delete, search, complete, upcoming	Track academic tasks
Attendance Manager	Record, calculate, warn, target calculation	Monitor attendance
Study Planner	Create, complete, delete, summary	Plan and monitor study sessions
Academic Performance	Store marks, total, grade	Track academic performance
Dashboard & Reports	Combined status and alerts	View overall academic information

5.2 Requirements Coverage

The supplied requirements are directly reflected in the documented feature set. Functional requirements map to the student, assignment, attendance, study, performance, and dashboard modules. Non-functional requirements are addressed through a menu-driven interface, input validation, modular organization, SQLite persistence, error handling, and lightweight use of standard-library components.

5.3 Testing and Expected Behaviour

The project includes automated unit tests. The supplied README states that the tests cover attendance calculations, performance calculations, and input-validation logic. Because no testoutput log was supplied with the project files, this report does not claim specific numerical test results or a test pass count.

5.4 Discussion

The project demonstrates a compact approach to academic information management. Its main design characteristic is integration: activities that are commonly handled separately are available through one application. The use of SQLite avoids the need for an external database server, while modular Python files make the project easier to maintain.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

CampusMate provides a lightweight command-line solution for managing several common student academic activities. The supplied project implements a menu-driven Python application backed by SQLite, with modules for student profile, assignments, attendance, study planning, academic performance, and dashboard reporting.

The project also addresses important software qualities described in the requirements: usability through a clear menu interface, reliability through validation, maintainability through modular code, persistence through SQLite, error handling through rejection of invalid values, and resource efficiency through local standard-library-based operation.

6.2 Future Enhancements

- Login and support for multiple student profiles.
- CSV export of academic information.
- Graphical user interface.
- Reminder notifications.
- Cloud synchronization.

References

- CampusMate – Student Productivity System, supplied README.md.
- CampusMate – Problem Statement and Scope, supplied statement.md.
- CampusMate project requirements, supplied requirements.txt.
- CampusMate main program, supplied main.py.
- Python standard library and unittest references as used by the project documentation.

Appendix – 1: Project Files and Execution

The supplied README describes Python 3 as the required runtime, recommends Python 3.10 or later, and states that no external packages are required. To run the application on Windows, the documented command is `python main.py`. Alternatives include `py main.py` or `python3 main.py` where appropriate.

The documented test command is: `python -m unittest discover -s tests -v`

Appendix – 2: Main Program Listing Summary

The supplied `main.py` imports the assignments, attendance, database, performance, reports, student, and study planner modules. It defines `display_banner()`, `main_menu()`, and `main()`. The `main()` function initializes the database and then starts the menu loop. The menu maps options 1–6 to the corresponding modules and option 7 to program exit.

Appendix – 3: Suggested Screenshots for Final Submission

- Main CampusMate menu screen
- Student Profile screen
- Assignment Manager screen
- Attendance calculation/warning screen
- Study Planner screen
- Academic Performance screen
- Dashboard & Reports screen
- SQLite data folder/database file
- Unit test execution output

Note: Actual screenshots are not included because they were not supplied with the source files. Insert screenshots from your own program execution in this appendix before final college submission.



CampusMate

Student Productivity System

Presented by: Hemanth Pentakota

Registration No.: 26MIM10061

Faculty/Supervisor: Dr. Adarsh Patel

VIT Bhopal University

Outline / Agenda

- Introduction
- Problem Statement
- Objectives
- Existing System / Literature Review
- Research Gap
- Development Methodology
- Functional & Non-Functional Requirements
- System Design (Architecture, Workflow, Diagrams)
- Work Plan
- Implementation
- Results and Discussion
- Testing
- Design Decisions, Challenges and Learnings
- Conclusion and Future Enhancements
- References

Introduction

- Students handle multiple academic activities – assignments, attendance, study schedules and performance tracking.
- This information is often scattered across notebooks, spreadsheets and separate apps.
- Manual tracking is inconvenient and makes it difficult to get a consolidated view of academic progress.
- CampusMate is a Python-based command-line application that brings these common academic management tasks into a single system.
- It is built using Python and SQLite, and is designed to be simple, modular and easy to extend.

Problem Statement

“Students often manage assignments, attendance, study schedules and academic performance separately, making it difficult to maintain an organized view of their academic activities. CampusMate provides a single Python-based system to manage these activities and generate a consolidated academic dashboard.”

Objectives

- To develop a simple Python-based student productivity system.
- To manage student academic information in one application.
- To provide assignment tracking and status management.
- To monitor attendance and identify low-attendance situations.
- To organize study sessions.
- To calculate academic performance and grades.
- To provide a consolidated dashboard.
- To demonstrate modular Python programming, database handling and testing.

Existing System / Literature Review

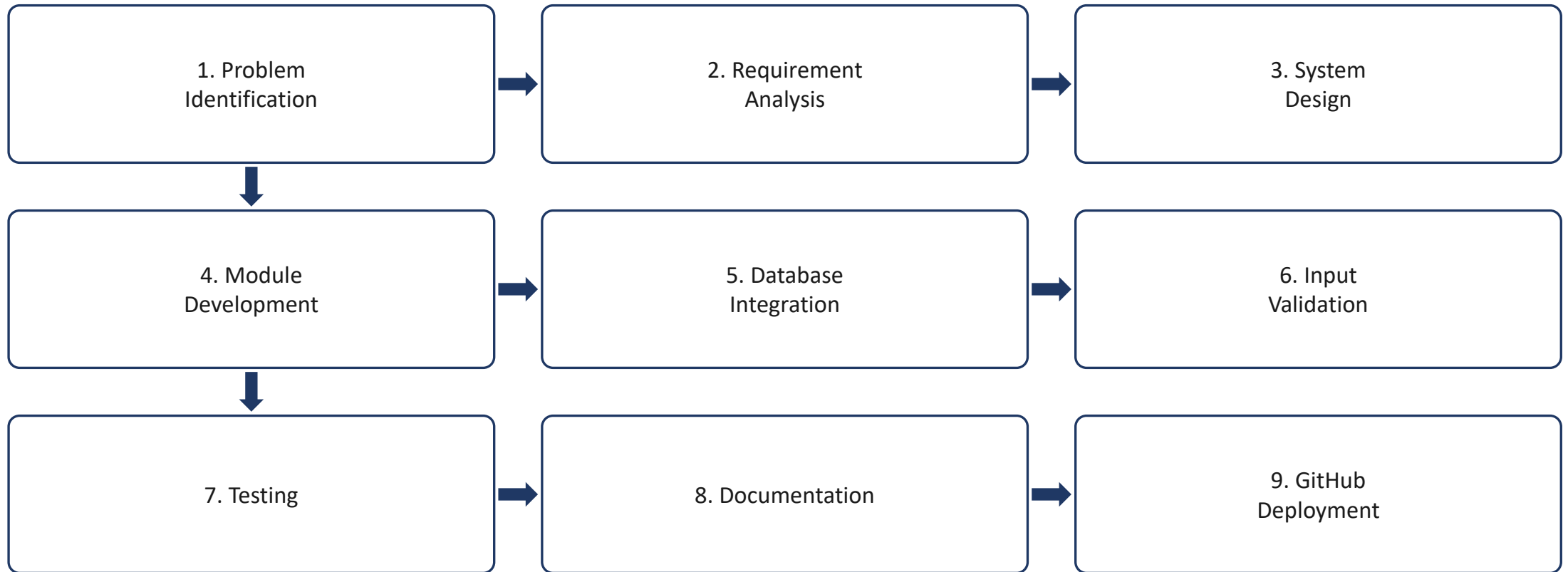
Category	Purpose	Advantages	Limitations
Manual academic tracking	Track assignments, attendance on paper	Simple, no tools needed	Error-prone, hard to consolidate
Spreadsheet-based tracking	Maintain records in Excel/Sheets	Flexible, widely available	Manual data entry, no automatic alerts
General task-management apps	Track tasks and to-do items	Good reminders and scheduling	Do not combine attendance or marks
Student productivity applications	Combine some academic features	Purpose-built for students	Often complex or not customizable

Gap: Existing approaches largely manage individual activities separately, while CampusMate combines student profile, assignments, attendance, study planning and performance into one lightweight Python application.

Research Gap / Existing System Gap

- Academic information may be distributed across multiple tools.
- General task applications may not combine attendance and marks.
- Manual tracking does not automatically calculate academic indicators.
- Students may need a lightweight solution rather than a complex enterprise platform.
- CampusMate combines multiple academic tracking functions in one modular application.
 - Note: CampusMate is not claimed to be the first or only application of its kind.

Development Methodology



Functional & Non-Functional Requirements

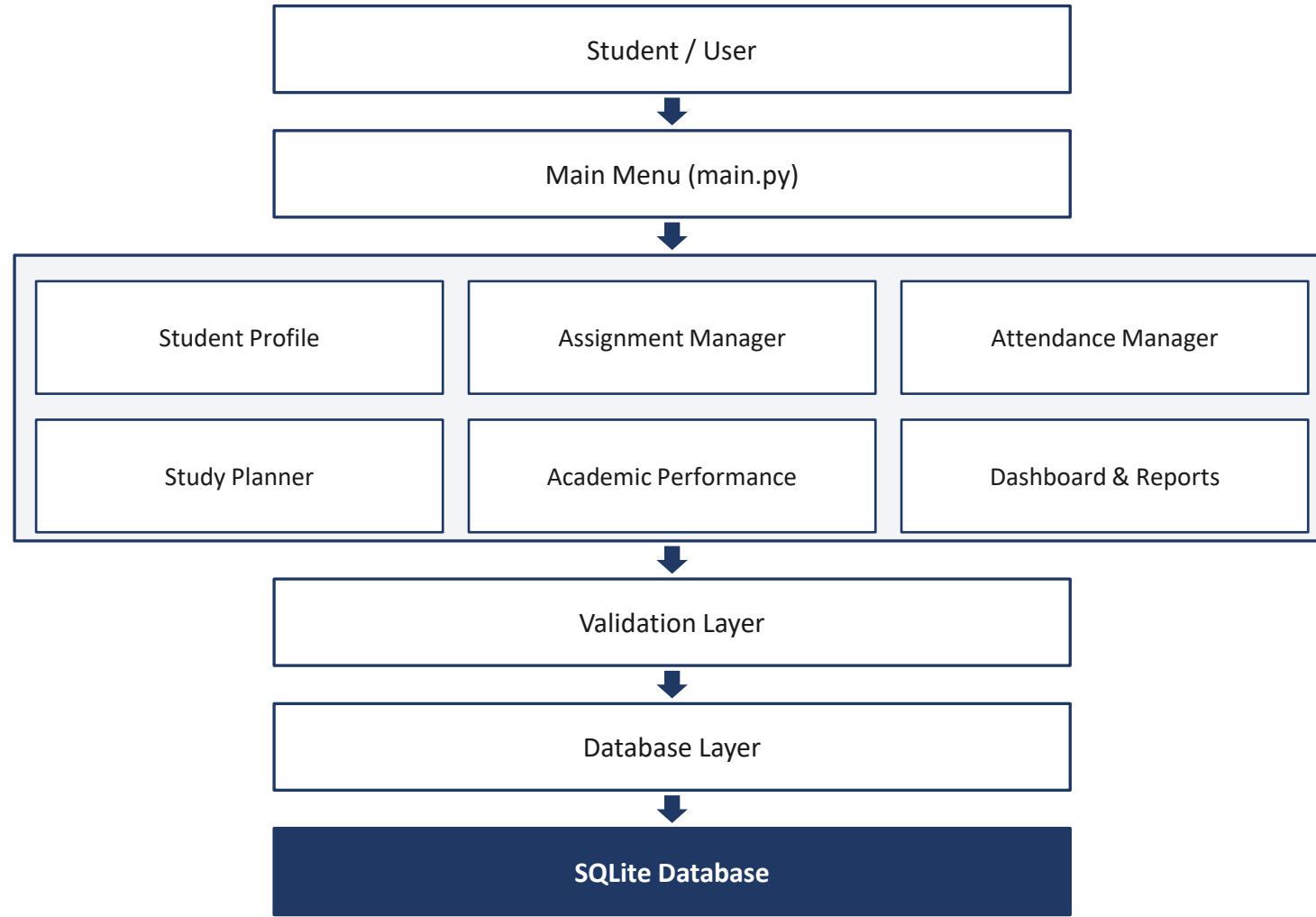
Functional Requirements

- FR1 – Student Profile Management
- FR2 – Assignment Management
- FR3 – Attendance Tracking
- FR4 – Study Planning
- FR5 – Academic Performance Management
- FR6 – Dashboard and Reporting
- FR7 – Data Validation
- FR8 – Persistent Data Storage

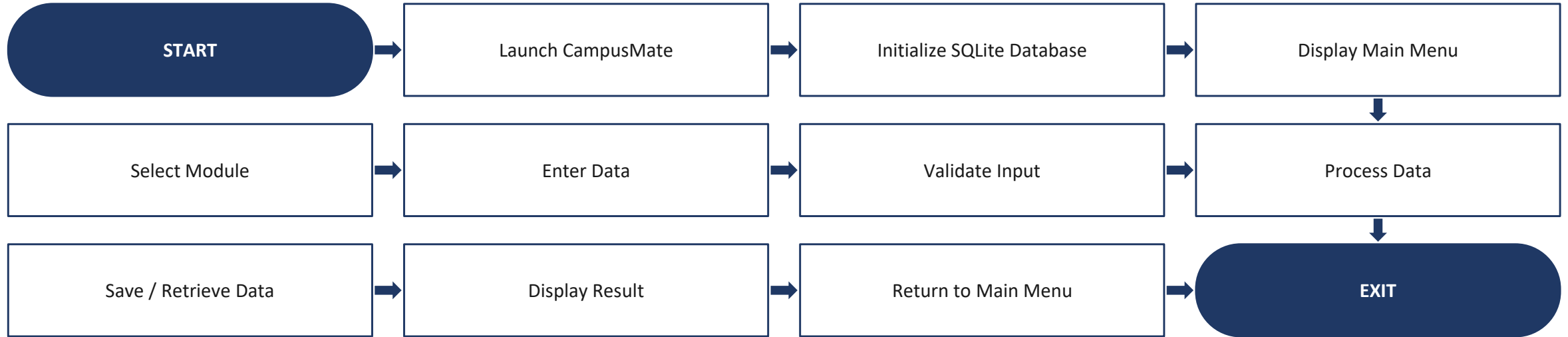
Non-Functional Requirements

- Usability – simple menu-driven interface
- Reliability – validates input, handles invalid entries
- Maintainability – divided into separate modules
- Data Persistence – SQLite storage
- Error Handling – invalid input does not crash the app
- Resource Efficiency – lightweight, local storage

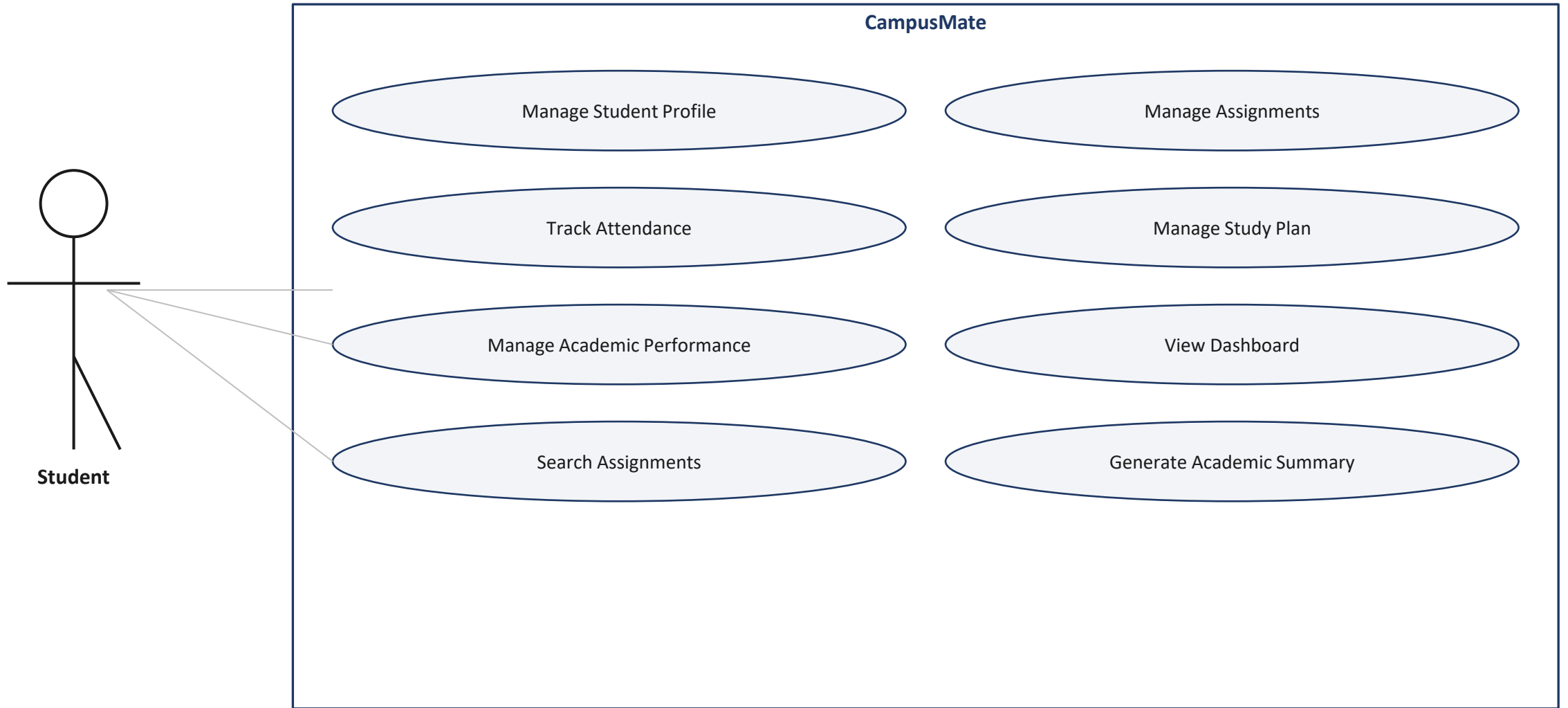
System Architecture



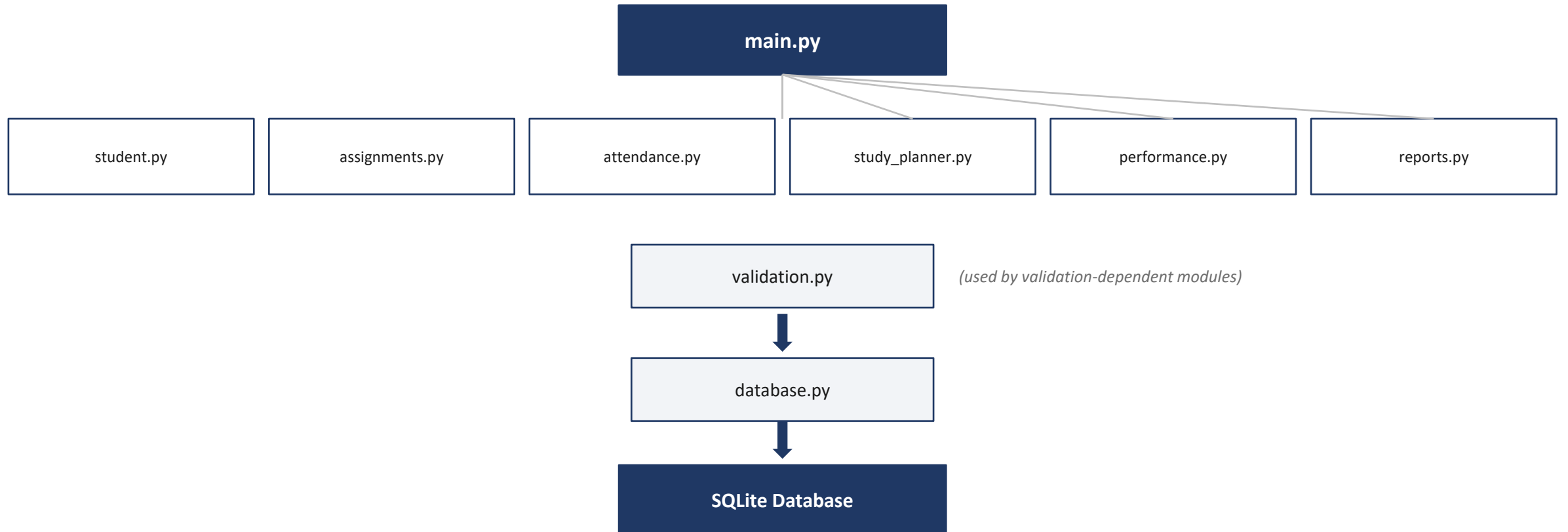
System Workflow



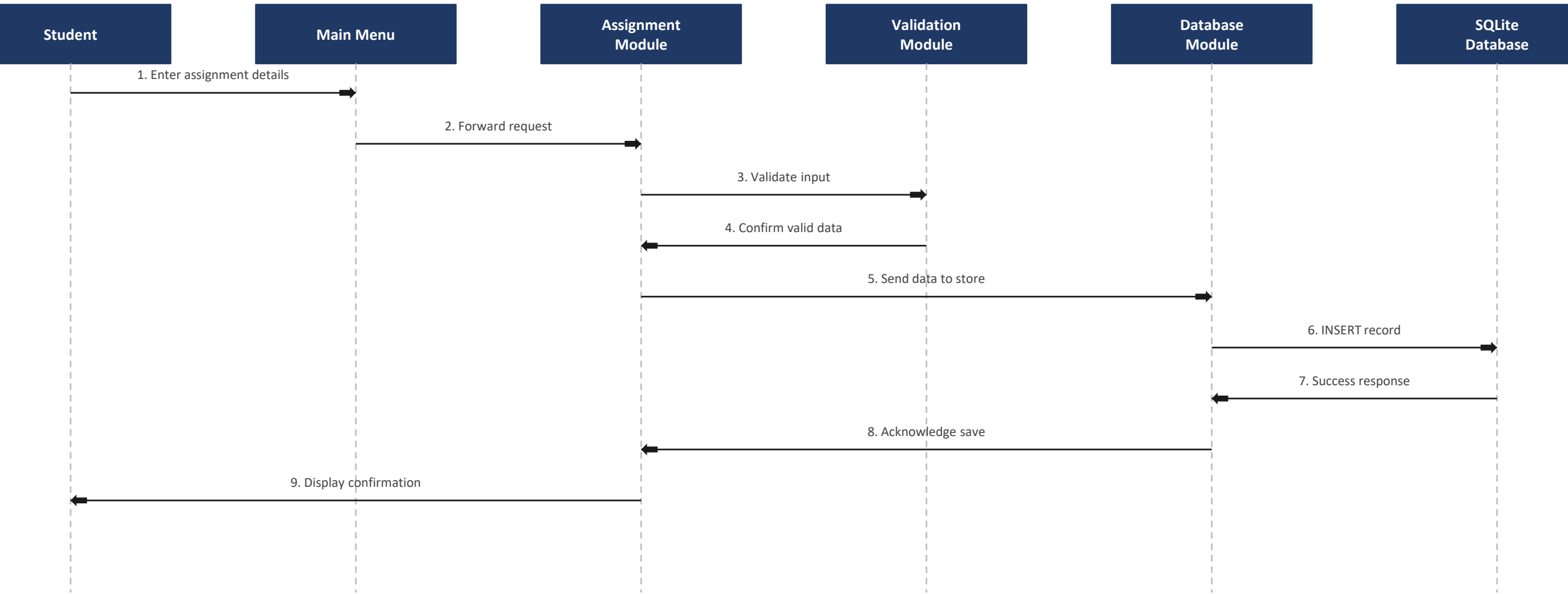
Use Case Diagram



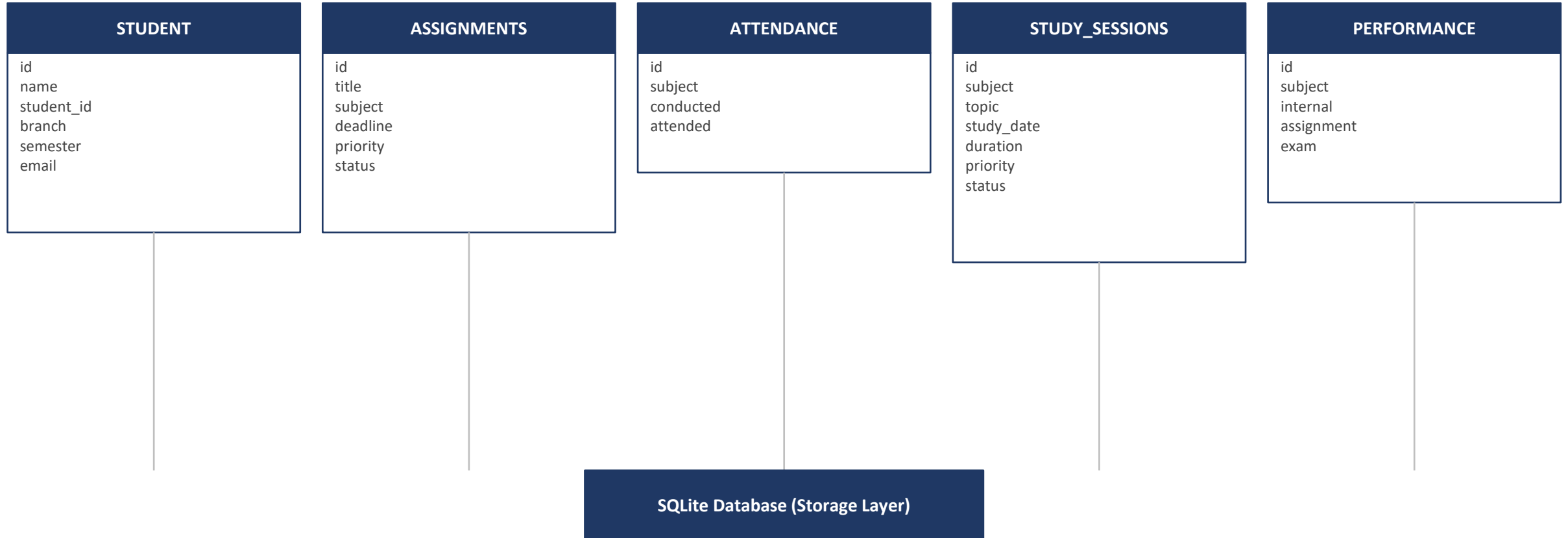
Component / Module Diagram



Sequence Diagram – Adding an Assignment



Database / ER Design



Note: Relationships shown reflect only what is implemented in the current project.

Work Plan

Week	Planned Activities
Week 1	Problem identification, requirement analysis, project planning
Week 2	Python project structure, Student Profile, Assignment Manager
Week 3	Attendance Manager, Study Planner, Academic Performance
Week 4	SQLite integration, Dashboard, Validation, Testing
Week 5	Documentation, diagrams, GitHub, final report, final testing

Implementation – Project Structure & Technologies

Python Project Structure

```
CampusMate/  
├── main.py  
├── README.md, requirements.txt  
├── src/  
│   ├── database.py  
│   ├── validation.py  
│   ├── student.py  
│   ├── assignments.py  
│   ├── attendance.py  
│   ├── study_planner.py  
│   ├── performance.py  
│   └── reports.py  
└── tests/  
    ├── test_attendance.py  
    ├── test_performance.py  
    └── test_validation.py
```

Core Technologies

Programming Language: Python

Database: SQLite

Testing: Python unittest

Dev. Environment: Visual Studio Code

Version Control: Git and GitHub

External Dependencies: None – standard library + SQLite

Implementation – Features ... *Contd.*

Implementation Features

- Menu-driven CLI
- Modular Python files
- SQLite persistence
- Input validation
- CRUD operations
- Calculations (attendance %, grades)
- Search functionality
- Reporting / dashboard
- Automated unit tests

Sample Code Snippet – Attendance Calculation

```
def attendance_percentage(attended, conducted):  
    if conducted == 0:  
        return 0.0  
    return (attended / conducted) * 100
```

Sample Output

```
>>> def attendance_percentage(attended: int, conducted: int) -> float:  
...     """Calculates the attendance percentage given classes attended and conducted.  
...     Handles division by zero by returning 0.0 if no classes were conducted.  
...     Cap the result at 100.0% if attended exceeds conducted due to input error.  
...     """  
...     if conducted == 0:  
...         return 0.0  
...     percentage = (attended / conducted) * 100  
...     return min(percentage, 100.0)  
...  
... print(attendance_percentage(20, 20))  
...  
100.0
```

Results and Discussion

Sample Input / Output – Attendance & Performance

Attendance

Classes Conducted	20
Classes Attended	14
Attendance %	70%
System Alert	Below 75% – Warning

Performance

Internal (max 30)	30
Assignment (max 10)	10
Exam (max 60)	50
Total / Grade	90/100 – Grade A+

```
1. Student Profile
2. Assignment Manager
3. Attendance Manager
4. Study Planner
5. Academic Performance
6. Dashboard & Reports
7. Exit

Enter your choice: 3

--- Attendance Manager ---
1. Add / Update Attendance
2. View Attendance
3. Back
Enter choice: 1
Subject: python
Classes conducted: 20
Classes attended: 14
Attendance saved. Current attendance: 70.00%.

--- Attendance Manager ---
1. Add / Update Attendance
2. View Attendance
3. Back
Enter choice: 2

=====
ATTENDANCE
=====
python Conducted: 20 Attended: 14 70.00% WARNING: Below 75% | Need 4 more present class(es)
=====
Overall Attendance: 70.00%
=====

--- Attendance Manager ---
1. Add / Update Attendance
2. View Attendance
3. Back
Enter choice: 1
```

```
1. Student Profile
2. Assignment Manager
3. Attendance Manager
4. Study Planner
5. Academic Performance
6. Dashboard & Reports
7. Exit

Enter your choice: 5

--- Academic Performance ---
1. Add / Update Performance
2. View Performance
3. Back
Enter choice: 1
Subject: python
Internal marks (0-30): 30
Assignment marks (0-10): 10
Exam marks (0-60): 50
Total: 90.00/100
Grade: A+
Performance saved successfully.

--- Academic Performance ---
1. Add / Update Performance
2. View Performance
3. Back
Enter choice: 2

=====
ACADEMIC PERFORMANCE
=====
python Internal: 30.0 Assignment: 10.0 Exam: 50.0 Total: 90.00 Grade: A+
=====
Overall Average: 90.00%
Overall Grade : A+
=====
```

Results and Discussion ... Contd.

Assignment (Sample)

Title	Python Essentials Project
Deadline	2026-09-30
Priority	High
Status	Pending

Dashboard: Combines student information, total/completed/pending assignments, overall attendance, academic average, overall grade, total study hours and academic alerts into a single consolidated view.

```
=====
CAMPUSWATE
Student Productivity System
=====

1. Student Profile
2. Assignment Manager
3. Attendance Manager
4. Study Planner
5. Academic Performance
6. Dashboard & Reports
7. Exit

Enter your choice: 2

--- Assignment Manager ---
1. Add Assignment
2. View Assignments
3. Update Assignment
4. Delete Assignment
5. Search Assignment
6. Mark Completed
7. Upcoming Assignments
8. Back
Enter choice: 1
Assignment title: Python Essentials Project
Subject: Python
Deadline (YYYY-MM-DD): 2026-09-30
Priority (Low/Medium/High): High
Assignment added successfully.

--- Assignment Manager ---
1. Add Assignment
2. View Assignments
3. Update Assignment
4. Delete Assignment
5. Search Assignment
6. Mark Completed
7. Upcoming Assignments
8. Back
Enter choice: 2

=====
ASSIGNMENTS
=====
ID: 1 | Python Essentials Project | Python | Due: 2026-09-30 | Priority: High | Status: Pending
ID: 2 | Python Essentials Project | Python | Due: 2026-09-30 | Priority: High | Status: Pending
```

Testing

Testing Framework: Python unittest

Test Areas

- Attendance percentage calculation
- Attendance target calculation
- Performance total calculation
- Grade calculation
- Email validation
- Date validation
- Input validation

12

Automated Tests

Expected Result: All tests pass successfully

```
PS C:\Users\Hemant\Downloads\Capsofute Starter\Capsofute> python main.py
=====
Capsofute
Student Productivity System
=====

1. Student Profile
2. Assignment Manager
3. Attendance Manager
4. Study Planner
5. Academic Performance
6. Dashboard & Reports
7. Exit

Enter your choice: 7

Thank you for using Capsofute.
Goodbye!

PS C:\Users\Hemant\Downloads\Capsofute Starter\Capsofute> python -m unittest discover -s tests -v
test_classroom (test_attendance.testAttendance.test_classroom) ... ok
test_full_attendance (test_attendance.testAttendance.test_full_attendance) ... ok
test_normal_attendance (test_attendance.testAttendance.test_normal_attendance) ... ok
test_new_classroom (test_attendance.testAttendance.test_new_classroom) ... ok
test_grade_a (test_performance.testPerformance.test_grade_a) ... ok
test_grade_a_plus (test_performance.testPerformance.test_grade_a_plus) ... ok
test_grade_full (test_performance.testPerformance.test_grade_full) ... ok
test_total (test_performance.testPerformance.test_total) ... ok
test_invalid_date (test_validation.testValidation.test_invalid_date) ... ok
test_invalid_email (test_validation.testValidation.test_invalid_email) ... ok
test_valid_date (test_validation.testValidation.test_valid_date) ... ok
test_valid_email (test_validation.testValidation.test_valid_email) ... ok

.....
Run 12 tests in 0.000s

OK
PS C:\Users\Hemant\Downloads\Capsofute Starter\Capsofute>
```

Design Decisions and Rationale

Why Python?

- Beginner-friendly syntax
- Standard library support
- Easy modular development

Why SQLite?

- Lightweight, local database
- No separate database server
- Suitable for a small student application

Why Modular Architecture?

- Easier maintenance
- Easier testing
- Separation of responsibilities

Why Command-Line Interface?

- Focuses on Python Essentials concepts
- Simple to implement
- Low resource requirements

Challenges and Key Learnings

Challenges Faced

- Organizing the project into modules
- Connecting Python modules with SQLite
- Validating different types of user input
- Handling incorrect user input
- Designing the database structure
- Testing different calculations
- Integrating multiple modules into one application

Key Learnings

- Python functions and modules
- Conditional statements and loops
- Exception handling
- Input validation
- CRUD operations
- SQLite database handling
- Software project organization
- Unit testing
- Git and GitHub
- Technical documentation

Conclusion and Future Enhancements

Conclusion

- CampusMate combines common student academic management activities in one application.
- The system demonstrates modular Python programming.
- SQLite provides persistent local data storage.
- Validation improves reliability.
- Dashboard functionality provides a consolidated academic view.
- Automated tests verify important calculations and validation logic.

Future Enhancements

(Not currently implemented)

- Graphical User Interface
- Web-based version
- Mobile application
- Cloud database
- Notification / reminder system
- Calendar integration
- Export reports to CSV/PDF
- Multi-student accounts
- Authentication
- More detailed analytics

References

- Python Documentation – <https://docs.python.org/3/>
- SQLite Documentation – <https://www.sqlite.org/docs.html>
- Python unittest Documentation – <https://docs.python.org/3/library/unittest.html>
- GitHub Documentation – <https://docs.github.com/>

Thank You!

Questions?

Hemanth Pentakota

Registration No.: 26MIM10061